

MIKROTIK • WIREGUARD • LARAVEL

ZoStream ISP: MikroTik + VPS WireGuard + Laravel Admin Panel Full Setup

VPS-hosted ISP admin panel-a MikroTik router pakhat emaw, tam zawk emaw, secure taka connect dan kimchang.

TESTED	RB5009UG+S+ • RouterOS 7.19.6 • Ubuntu VPS
NETWORK	WireGuard 10.77.0.0/24 • VPS UDP 51820
UPDATED	13 July 2026

[Security-first production guide](#)

Contents

1. Final network design	3
2. Information to prepare	4
3. VPS: install WireGuard	4
4. VPS: generate a safe key pair	4
5. MikroTik: create WireGuard interface in Winbox	4
6. MikroTik: assign tunnel IP	5
7. VPS: create /etc/wireguard/wg0.conf	5
8. VPS: open UDP 51820	6
9. VPS: start WireGuard	6
10. MikroTik: add the VPS peer in Winbox	6
11. Verify the WireGuard tunnel	7
12. MikroTik: create a restricted REST user group	7
13. MikroTik: create the REST API user	7
14. MikroTik: enable REST HTTP only for the VPS	8
15. MikroTik: add and correctly order the firewall rule	8
16. Test RouterOS REST from the VPS	8
17. Laravel production configuration	9
18. Add MikroTik from the Laravel admin panel	9
19. Confirm the current FreeRADIUS admin workflow	10
20. Configure additional MikroTik routers	11
21. Troubleshooting decision table	12
22. Production security checklist	13
23. Safe rollout and cleanup	13

Tested environment: MikroTik RB5009UG+S+, RouterOS 7.19.6, Ubuntu VPS, Laravel admin panel at `https://isptest.zostream.in`

This document explains how to connect one or more MikroTik routers to a Laravel admin panel hosted on a public VPS. The recommended design uses a private WireGuard tunnel. MikroTik REST port 80 is reachable only from the VPS tunnel IP and is not exposed to the public internet.

Current production authentication: Subscribers are authenticated by central FreeRADIUS using the same MySQL database as Laravel. REST is retained for router health, active-session discovery and disconnect actions. The complete, current RADIUS and PPPoE procedure is in RADIUS_CUTOVER_SETUP.md; that guide is authoritative if this older WireGuard/REST walkthrough conflicts with it.

Important: Never paste a WireGuard private key or production password into chat, email, screenshots, source control, or this document. If a private key or temporary password has already been shared, rotate it before production use.

1. Final network design

```
TEXT
Laravel admin / API on VPS
Public IP: 153.92.223.7
WireGuard: 10.77.0.1/24, UDP 51820
    |
    | encrypted WireGuard tunnel
    |
MikroTik router 1: 10.77.0.2/24
MikroTik router 2: 10.77.0.3/24
MikroTik router 3: 10.77.0.4/24
```

The Laravel server connects to router 1 using `http://10.77.0.2/rest/...` HTTP is acceptable inside this design because the traffic is encrypted by WireGuard. Do not expose MikroTik port 80 to the public internet.

FreeRADIUS and REST have different jobs:

- FreeRADIUS authenticates PPPoE subscribers and returns package speed/accounting attributes.
- REST checks router health, reads current PPP sessions and disconnects suspended/deleted sessions.
- Both paths stay private inside WireGuard.

2. Information to prepare

Item	Router 1 example
VPS public IP	153.92.223.7
VPS WireGuard IP	10.77.0.1/24
VPS WireGuard UDP port	51820
MikroTik tunnel IP	10.77.0.2/24
MikroTik WireGuard interface	wg-vps
REST user	zostream-api
REST port	80
RADIUS server	10.77.0.1
RADIUS ports	1812/udp, 1813/udp
Router RADIUS source	the router's own WireGuard IP

Choose a different tunnel IP for every additional router. Never reuse a WireGuard private/public key pair between routers.

3. VPS: install WireGuard

SSH into the VPS and run:

```
BASH
sudo apt update
sudo apt install -y wireguard
```

Confirm installation:

```
BASH
wg --version
```

4. VPS: generate a safe key pair

Generate the VPS private key with restrictive permissions and derive the public key:

```
BASH
sudo sh -c 'umask 077; wg genkey > /etc/wireguard/vps-private.key'
sudo sh -c 'wg pubkey < /etc/wireguard/vps-private.key > /etc/wireguard/vps-public.key'
sudo chmod 600 /etc/wireguard/vps-private.key
sudo chmod 644 /etc/wireguard/vps-public.key
```

Display only the public key:

```
BASH
sudo cat /etc/wireguard/vps-public.key
```

Copy this value as `VPS_PUBLIC_KEY`. The MikroTik peer needs it. Do not copy or share `/etc/wireguard/vps-private.key`.

If an old VPS private key was exposed, generate a new pair, replace the private key in `wg0.conf`, and replace the VPS public key on every MikroTik peer.

5. MikroTik: create WireGuard interface in Winbox

1. Open **Winbox** → **WireGuard** → **WireGuard**.

2. Click **New (+)**.
3. Set **Name** to `wg-vps`.
4. Set **MTU** to `1420`.
5. A MikroTik listen port such as `13231` may be kept. The router is the outbound peer, so this port does not need public port forwarding.
6. Leave **Private Key** empty and click **Apply**. RouterOS generates the interface key pair.
7. Copy the generated **Public Key**. This is `MIKROTIK_PUBLIC_KEY` for the VPS peer.
8. Never copy the MikroTik private key into the VPS configuration.

The interface public key and peer public key are different:

- MikroTik **interface** public key goes into the VPS `[Peer]` section.
- VPS public key goes into the MikroTik **peer** Public Key field.

6. MikroTik: assign tunnel IP

1. Open **Winbox** → **IP** → **Addresses**.
2. Click **New (+)**.
3. Set **Address** to `10.77.0.2/24`.
4. Set **Interface** to `wg-vps`.
5. Click **OK**.

Optional verification in **New Terminal**:

```
ROUTEROS
/ip address print where interface=wg-vps
```

7. VPS: create `/etc/wireguard/wg0.conf`

Open the file:

```
BASH
sudo nano /etc/wireguard/wg0.conf
```

Use this configuration, replacing both key placeholders with actual values:

```
INI
[Interface]
Address = 10.77.0.1/24
ListenPort = 51820
PrivateKey = VPS_PRIVATE_KEY

[Peer]
# MikroTik router 1
PublicKey = MIKROTIK_PUBLIC_KEY
AllowedIPs = 10.77.0.2/32
```

PrivateKey must contain the value from `/etc/wireguard/vps-private.key`. Keep the file protected:

```
BASH
sudo chmod 600 /etc/wireguard/wg0.conf
```

AllowedIPs must be `/32` per router on the VPS. Do not assign the same IP to two peers.

8. VPS: open UDP 51820

If UFW is active:

```
BASH
sudo ufw allow 51820/udp
sudo ufw status
```

Also allow inbound UDP 51820 in the VPS provider firewall/security group. UFW alone is not enough when the hosting provider has a separate firewall.

No TCP port 80 rule is required for the MikroTik REST connection on the VPS public interface.

9. VPS: start WireGuard

```
BASH
sudo systemctl enable --now wg-quick@wg0
sudo systemctl restart wg-quick@wg0
sudo systemctl status wg-quick@wg0 --no-pager
sudo wg show
```

At this point, `wg show` lists the interface and configured peer. `latest handshake` appears only after the MikroTik peer is configured and sends traffic.

The current VPS public key can always be verified with:

```
BASH
sudo wg show wg0 public-key
```

Use this current value on MikroTik. Do not use an older key copied before `wg0.conf` was changed.

10. MikroTik: add the VPS peer in Winbox

1. Open **Winbox** → **WireGuard** → **Peers**.
2. Click **New (+)**.
3. Set these fields:

Field	Value
Name	vps-peer
Interface	wg-vps
Public Key	VPS_PUBLIC_KEY
Private Key	leave blank
Endpoint Address	153.92.223.7
Endpoint Port	51820
Allowed Address	10.77.0.1/32
Persistent Keepalive	25 seconds
Responder	OFF
Preshared Key	leave blank unless configured identically on both sides

4. Click **Apply**, then **OK**.

The Endpoint Address is the VPS public IP, not 10.77.0.1. The Allowed Address is the VPS tunnel IP, not the MikroTik IP.

11. Verify the WireGuard tunnel

On MikroTik **WireGuard** → **Peers**, verify:

- **Last Handshake** is recent, not 00:00:00.
- **Rx** and **Tx** are greater than zero.
- **Current Endpoint Address** shows the VPS public IP.

On the VPS:

```
BASH
sudo wg show
ping -c 3 10.77.0.2
```

Expected `wg show` peer details include:

```
TEXT
endpoint: <router-public-ip>:<port>
allowed ips: 10.77.0.2/32
latest handshake: ... seconds ago
transfer: ... received, ... sent
```

Do not continue to Laravel setup until the handshake and ping are working.

12. MikroTik: create a restricted REST user group

The tested RouterOS 7 policy set is:

```
ROUTEROS
/user group add name=isp-panel policy=read,write,web,api,rest-api
```

If the group already exists:

```
ROUTEROS
/user group set [find where name="isp-panel"] policy=read,write,web,api,rest-api
```

Verify:

```
ROUTEROS
/user group print detail where name="isp-panel"
```

The earlier minimal `read,write,rest-api` policy can return 401 Unauthorized on this RouterOS setup. Do not leave the API user in the `full` group after testing.

13. MikroTik: create the REST API user

Using Winbox:

1. Open **System** → **Users** → **Users**.
2. Click **New (+)**.
3. Set **Name** to `zostream-api`.
4. Set **Group** to `isp-panel`.
5. Set **Allowed Address** to `10.77.0.1/32`.
6. Set a long, unique production password and store it in a password manager.
7. Click **OK**.

Equivalent Terminal command:

```
ROUTEROS
/user add name=zostream-api group=isp-panel address=10.77.0.1/32 password="REPLACE_WITH_STRONG_PASSWORD"
```

If the user already exists:

```
ROUTEROS
/user set [find where name="zostream-api"] disabled=no group=isp-panel address=10.77.0.1/32
password="REPLACE_WITH_STRONG_PASSWORD"
```

Verify:

```
ROUTEROS
/user print detail where name="zostream-api"
```

The Allowed Address must be the VPS tunnel IP `10.77.0.1/32`, not the VPS public IP and not the old LAN IP.

14. MikroTik: enable REST HTTP only for the VPS

1. Open **Winbox** → **IP** → **Services**.
2. Open the `www` service.
3. Enable it.
4. Set **Port** to `80`.
5. Set **Available From / Address** to `10.77.0.1/32`.
6. Click **OK**.

Equivalent Terminal command:

```
ROUTEROS
/ip service set [find where name="www"] disabled=no port=80 address=10.77.0.1/32
```

Verify:

```
ROUTEROS
/ip service print detail where name="www"
```

15. MikroTik: add and correctly order the firewall rule

Add a rule that accepts REST traffic only from the VPS tunnel:

```
ROUTEROS
/ip firewall filter add chain=input action=accept protocol=tcp in-interface=wg-vps src-address=10.77.0.1/32 dst-port=80 comment="Allow VPS REST"
```

Print the rule list:

```
ROUTEROS
/ip firewall filter print stats without-paging
```

The `Allow VPS REST` rule must be above `defconf: drop all not coming from LAN`. Firewall rules run from top to bottom. A correct rule below the drop rule receives zero packets and the VPS curl request times out.

Move the rule above the drop rule. In Winbox, drag it above the drop rule, or use the terminal after checking rule numbers:

```
ROUTEROS
/ip firewall filter move [find where comment="Allow VPS REST"] [find where comment="defconf: drop all not coming from LAN"]
```

Verify again and confirm the allow rule packet counter increases when curl is run.

16. Test RouterOS REST from the VPS

Run:

```
BASH
```

```
curl --basic --user 'zostream-api:REPLACE_WITH_STRONG_PASSWORD' \
  http://10.77.0.2/rest/system/resource
```

Successful output is JSON similar to:

```
JSON
{
  "architecture-name": "arm64",
  "board-name": "RB5009UG+S+",
  "platform": "MikroTik",
  "version": "7.19.6 (stable)"
}
```

Use `--basic` explicitly during diagnosis. If curl prompts for the password, enter the API user's current password exactly.

17. Laravel production configuration

On the VPS, go to the deployed project:

```
BASH
cd /var/www/isptest.zostream.in
```

Confirm the production `.env` has the correct application URL, database, admin credentials, and API token. Example variable names used by this project:

```
DOTENV
APP_ENV=production
APP_DEBUG=false
APP_URL=https://isptest.zostream.in
ADMIN_EMAIL=your-admin@example.com
ADMIN_PASSWORD=REPLACE_WITH_A_STRONG_ADMIN_PASSWORD
ISP_API_TOKEN=REPLACE_WITH_A_LONG_RANDOM_TOKEN
```

Then run the normal Laravel deployment steps:

```
BASH
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan optimize
```

The current migrations include the FreeRADIUS user, accounting, NAS and group-query tables. Confirm that `radcheck`, `radreply`, `nas`, `radacct`, `radpostauth`, `radusergroup`, `radgroupcheck` and `radgroupreply` exist before enabling subscriber authentication.

Run `php artisan migrate --seed --force` only when intentionally creating/updating the seeded admin account. Change the default `admin@example.com` / `change-me-now` credentials before production use.

`APP_KEY` encrypts saved router passwords. Back it up securely and do not rotate it after router credentials have been saved unless a planned re-encryption migration is performed.

18. Add MikroTik from the Laravel admin panel

1. Open `https://isptest.zostream.in`.
2. Sign in with the production admin email and password.
3. Open **Routers** → **Add router**.
4. Enter:

Admin field	Router 1 value
Display name	Main MikroTik
Router IP / hostname	10.77.0.2

Admin field	Router 1 value
REST API port	80
API username	zostream-api
API password	the current API password
Use HTTPS	OFF
Verify TLS certificate	OFF
Router is active	ON
Enable RADIUS	ON
RADIUS shared secret	unique 16–60 character secret for this router

- Click **Save router**.
- On the router list, click **Test**.
- In MikroTik **RADIUS**, use the exact same shared secret, server `10.77.0.1`, service `ppp`, and this router's WireGuard IP as **Src. Address**.
- Restart FreeRADIUS after adding/changing the router so `read_clients = yes` reloads the `nas` table.

Expected success message:

TEXT

Connected to RB5009UG+S+ (RouterOS 7.19.6 (stable)).

The admin panel stores the router password encrypted using Laravel's application key.

19. Confirm the current FreeRADIUS admin workflow

After the router REST test and RADIUS setup succeed:

- Open **Packages** and create a package with a RouterOS-compatible rate such as `10M/10M`.
- Open **Customers** → **Add customer**.
- Select the correct router and package.
- Enter a globally unique PPPoE username, password, Active status and future expiry.
- Save. Laravel automatically writes the password/status to `radcheck` and package speed/accounting interval to `radreply`.
- Configure the customer ONU/ONT/CPE with the same case-sensitive credentials.
- Confirm MikroTik RADIUS **Accepts** increases and the user appears under **PPP** → **Active Connections**.
- Suspend the customer and verify the active session disconnects and the next login is rejected.
- Renew/reactivate and verify the next login succeeds.
- Change the package, verify `radreply.Mikrotik-Rate-Limit` changes and reconnect to apply the new speed.

Normal customer work does not create or update local MikroTik PPP Secrets. The customer **Sync** action is retained for RADIUS backfill/recovery. Legacy local secrets must be removed in controlled batches after successful tests because RouterOS checks a matching local secret before RADIUS.

20. Configure additional MikroTik routers

Use one tunnel subnet with a unique IP and peer key for every router:

Router	MikroTik tunnel IP	VPS AllowedIPs
Router 1	10.77.0.2/24	10.77.0.2/32
Router 2	10.77.0.3/24	10.77.0.3/32
Router 3	10.77.0.4/24	10.77.0.4/32

For router 2, append this to the VPS `wg0.conf`:

```
INI
[Peer]
# MikroTik router 2
PublicKey = ROUTER_2_PUBLIC_KEY
AllowedIPs = 10.77.0.3/32
```

Then:

```
BASH
sudo systemctl restart wg-quick@wg0
sudo wg show
ping -c 3 10.77.0.3
```

On router 2, repeat the Winbox steps with:

- Interface address `10.77.0.3/24`.
- The same VPS public key, public endpoint `153.92.223.7`, UDP `51820`.
- Peer Allowed Address `10.77.0.1/32` and `keepalive 25`.
- REST user Allowed Address `10.77.0.1/32`.
- Service and firewall restricted to `10.77.0.1/32`.
- Prefer a different strong REST password for each router.
- Add a unique RADIUS shared secret in the admin panel and MikroTik.
- Set MikroTik RADIUS **Src. Address** to `10.77.0.3`.
- Restart FreeRADIUS after the `nas` row is added or changed.

Finally, add router 2 to Laravel with host `10.77.0.3`. The project supports multiple router records; each customer is assigned to one router, while central package rates are written to each assigned subscriber's RADIUS reply row.

21. Troubleshooting decision table

Symptom	Meaning	Fix
Last Handshake 00:00:00, Rx/Tx 0	WireGuard not connected	Check current VPS public key, endpoint IP/port, provider firewall UDP 51820, keepalive 25
ping 10.77.0.2 fails	Tunnel/routing issue	Verify interface addresses, peer AllowedIPs, duplicate IPs, and <code>wg show</code>
curl timeout / could not connect	REST service or input firewall blocked	Enable <code>www</code> , restrict it to <code>10.77.0.1/32</code> , move allow rule above WAN drop
curl returns 401 Unauthorized	Network works; authentication/authorization failed	Reset password, verify username, <code>isp-panel</code> policies, user Allowed Address
Browser shows MikroTik login UI	Web service is reachable	This is WebFig; the Laravel panel still uses <code>/rest</code> with Basic Auth
Rule packet counter remains 0	Traffic does not reach that rule	Check rule order, interface/source match, and destination address
Laravel Test fails but VPS curl works	Saved panel configuration differs	Check host, port, username/password, HTTP/HTTPS switches; clear config cache if <code>.env</code> changed
Handshake stops after a while behind NAT	Router mapping expired	Set Persistent Keepalive to 25 seconds on MikroTik
RADIUS Requests stay at 0	PPPoE login never reached RADIUS	Check CPE attempt, PPPoE server interface/service, Use RADIUS, local PPP Secret priority and OLT/VLAN path
Requests rise and Timeouts rise	FreeRADIUS did not reply	Check UDP 1812 on <code>wg0</code> , NAS source IP, <code>read_clients = yes</code> , shared secret and FreeRADIUS restart
Correct password still Access-Rejects	SQL authorize failed or account is rejected	Inspect <code>radcheck</code> , <code>radpostauth</code> and <code>freeradius -X</code> ; migrate missing group tables
Access-Accept but no PPP Active session	PPP negotiation/profile failed	Check default profile local address, remote pool, PPP logs and CPE behavior
PPP Active exists but no internet	Routing/NAT/DNS problem	Check assigned IP, default route, masquerade/routed pool and PPP DNS

Useful commands:

```
BASH
sudo wg show
ip route get 10.77.0.2
ping -c 3 10.77.0.2
curl -v --basic --user 'zostream-api:PASSWORD' http://10.77.0.2/rest/system/resource
sudo journalctl -u wg-quick@wg0 -n 100 --no-pager
```

MikroTik verification commands:

```
ROUTEROS
/interface/wireguard print detail
/interface/wireguard/peers print detail
/ip/address print where interface=wg-vps
/user print detail where name="zostream-api"
/user group print detail where name="isp-panel"
/ip service print detail where name="www"
/ip firewall filter print stats without-paging
```

22. Production security checklist

- Rotate any private key or password previously exposed in chat or screenshots.
- Keep the VPS private key readable only by root.
- Keep MikroTik REST and user Allowed Address at `10.77.0.1/32`.
- Keep the REST firewall rule above the WAN drop rule.
- Never expose MikroTik TCP 80 directly to `0.0.0.0/0`.
- Do not use the `admin` or `full` group account from Laravel.
- Use a unique API password for each router.
- Restrict Winbox to management networks/tunnel addresses separately.
- Keep RouterOS, Ubuntu, PHP, Laravel dependencies, and the web server patched.
- Use HTTPS for the public Laravel admin panel.
- Keep RADIUS UDP 1812/1813 reachable only over `wg0` from the router subnet.
- Keep MySQL private; FreeRADIUS uses recoverable PPP credentials in `radcheck`.
- Add login throttling before broad public exposure of the admin panel.
- Run Laravel `schedule:run` every minute so the midnight expiry command executes.
- Back up `/etc/wireguard/wg0.conf`, Laravel `.env`, `APP_KEY`, and the database securely.
- Test restoring the backup before relying on it.
- Review `/user active`, WireGuard peer endpoints, and firewall counters periodically.

23. Safe rollout and cleanup

1. Confirm WireGuard handshake and VPS ping to the MikroTik tunnel IP.
2. Confirm REST curl returns system JSON and Laravel **Test** succeeds.
3. Confirm FreeRADIUS SQL/NAS configuration and zero RADIUS Timeouts/Bad Replies.
4. Confirm one test customer receives Access-Accept, a PPP address, internet and its package speed.
5. Confirm suspend, renew, expiry and accounting behavior.
6. Remove migrated local PPP Secrets only in small verified batches.
7. Disable obsolete LAN/public REST rules and old temporary accounts.
8. Rotate every credential/key exposed during setup and make a final tested backup.

Following this order prevents accidental lockout and leaves a clear rollback path.